```
(function() {
  var Comments;
  Comments = (function() {
    function Comments() {}
    return Comments;
  })();
  exports.Comments = Comments;
}).call(this);
```

The result of compiling main.coffee

```
(function() {
  var Comments;
  Comments = require('comments').Comments;
}).call(this);
```

The result of compiling comments.coffee

This won't work in any browser. In order to allow modules to `require` other modules, you need to write your own module system or use an existing solution.

Client-side module solutions

There are many alternatives and they differ per platform. The solution presented here is along the same lines as Stitch; see https://github.com/sstephenson/stitch/.

If you write your own module system, then you'll have to change how your CoffeeScript program is compiled. That's not so scary after chapter 8, so you decide to write your own—with a little help from Agtron.

12.3.2. How to write a module system

For modules to work when they're all contained in a single file, there must be some way to define a module and to tell it how to do `require` and `exports`. On the server, a module was defined implicitly by just having a file. On the client, however, the individual files will be concatenated to a single file, with each module declared and passed to a `defmodule` function, as shown in the following JavaScript example: